- Introduzione
  - esame finale:
- Concetti fondamentali
  - qualità del software
  - Processi software
    - Metodi e strumenti
- Attività fondamentali dei processi software
  - STUDIO DI FATTIBILITÀ:
  - CODE AND FIX
  - Acquisizione, analisi e specifica dei requisiti
  - Progettazione e Sviluppo
  - Verifica e Validazione
    - Test dei componenti
    - Test del sistema
    - Test del cliente
    - Test iterativo
  - Evoluzione
- Modelli di processi software
  - Modello a cascata
    - Modello a cascata con retroazione
    - Modello a cascata a V
  - Modelli evolutivi

Introduzione

Nota:
appunti tratti dalle lezioni e slides del corso di Ingegneria del Software, tenuto dal prof. Riccio Vincenzo

si inizia di solito alle 8:45
l'anno scorso tutti sono passati tra i vari appelli
materiale didattico su elearning - corso 725

Ricevimento:

- lunedì 15:00-17:00 - L2-02-BC (A2 129)
- volendo anche su Teams

esercitazioni in gruppo (2-3 persone), identiche a quelle del esame, si faranno a lezione e poi avverrà un confronto alla fine della lezione

libro: Ingegneria del software, Ian Sommerville, prima edizione

esame finale:

- scritto carta e penna (120 minuti)
- 3 parti:
  - esercizi su analisi e modellazione
  - esercizi di testing
  - domande di teoria

tutte e 3 le parti devono risultare sufficienti per superare l'esame

orale solo per la lode

Concetti fondamentali

cosa è software?

- è un termine che indica l'insieme dei programmi per computer e la relativa **documentazione** (anche i documenti legali)

noi terremo conto anche di tutta la documentazione prodotta durante lo sviluppo del software, come modelli di progetto, manuali utente, siti web di supporto, ecc.

definizione IEEE:
"software è l'insieme di programmi, procedure, regole e documentazione relativa al funzionamento di un sistema di elaborazione dati"

"software is eating the world" - Marc Andreessen, 2011
si sta diffondendo in tutti i settori, non solo in informatica con una capitalizzazione sempre più grande (ha superato il 1468 miliardi)
con gli LLM non è cambiato il concetto, solo la velocità in cui il software viene sviluppato. Che ci piaccia o no, gli LLM rimangono software e "programmare" con essi cambia solo il livello di linguaggio usato per lo sviluppo (linguaggio naturale).

Software 1.0:
vedi il problema, pianifichi un algoritmo, progetti un algoritmo e implementi l'algoritmo
Software 2.0:
prepari i dati, definisci un modello, addestri il modello e valuti il modello
Software 3.0:
prepara i dati, definisci un modello, addestri il modello, valuti il modello - iterativamente (con un transformer)

software generico:

- sw prodotto autonomamente da una organizzazione per incontrare le necessità di svariati clienti
- il produttore ha controllo sulle specifiche del sw

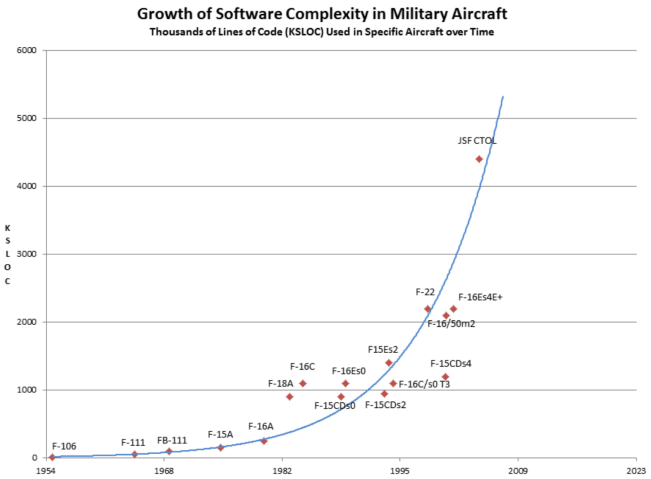
software custom:

- sw sviluppato da una organizzazione su commissione di uno specifico cliente
- il produttore deve attenersi alle specifiche indicate dal cliente

qualità del software

è importante tenere conto della complessità, perché è fondamentale che un software sia **manutenibile**
Statisticamente, la manutenzione del codice è l'aspetto più costoso del ciclo di vita del software, quindi risolvere eventuali problemi durante la scrittura riduce drasticamente costi e tempi in fase di manutenzione.
se si scopre un problema:

- durante la scrittura: costo 1
- durante la rilettura: costo 7
- durante il testing: costo 20
- dopo la consegna: costo 100



Fonte: SAVI-AVSI (2023)

Il numero di linee di codice del software sta crescendo esponenzialmente
Nonostante la maggiore complessità, la velocità con cui il software sta venendo sviluppato è aumentata nel tempo, evento collegato sia a strumenti migliori per scriverlo, sia allo sviluppo di approcci di ingegneria del software
il successo o l'insuccesso di un progetto software è spesso legato all'applicazione dell'ingegneria del software per garantire la qualità del prodotto (anche se nel breve termine il codice potrebbe essere considerato "migliore")
un codice facilmente mantenibile rende la scrittura di nuovo codice più veloce (e ad un certo punto diventa impossibile scrivere nuovo codice senza questo processo)

Sviluppo personale	Sviluppo professionale
• Difficilmente il software sarà riusato in futuro da altri utenti • Non è necessario scrivere una guida, né un documento di progetto • I requisiti di qualità della documentazione sono meno stringenti	• Il software sarà usato da altre persone diverse dai propri sviluppatori • Sviluppo in team e, quindi, probabilmente il software sarà modificato e manutenuto da altri sviluppatori • Il software deve possedere caratteristiche di qualità

Le caratteristiche qualitative di un software professionale sono:

- **Accettabilità**
- **Fidatezza e Protezione**
- **Efficienza**
- **Mantenibilità**

L’insieme specifico dei caratteri qualitativi di un sistema software dipende dalla sua applicazione

Sviluppare software sta diventando sempre più costoso, perché le operazioni che un software deve compiere sono sempre più complesse (siamo passati da giocare a Pong a creare macchine a guida autonoma)
questo fenomeno è stato cercato di risolvere nel 1968 alla NATO Software Engineering Conference di Garmisch, dove il problema venne definito come "**software crisis**"
La soluzione: **ingegneria del software**, che poi fece nascere la **IEEE** (Institute of Electrical and Electronics Engineers), ovvero applicare i principi dell'ingegneria ai processi di sviluppo del software

"L’Ingegneria del Software
è una disciplina
ingegneristica che
riguarda tutti gli aspetti
della produzione del software"

L’Ingegneria del Software (IS) è una disciplina ingegneristica (è l'intersezione tra ingegneria e informatica)

Processi software

Processo software: insieme di attività che porta alla creazione o all'evoluzione di un prodotto software

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

Metodi e strumenti

Metodi:

- Approcci strutturati per sviluppare software di qualità, a costi contenuti ed entro i tempi di consegna
- Tali metodi forniscono una guida alle attività dei processi e alla relativa organizzazione

Non sono i comandamenti, ma solide linee guida sviluppate nei corsi di decenni

strumenti:

- Sistemi software usati per aiutare le attività dei processi software (ad es., analisi, modellazione, debugging, testing)

ALCUNE SFIDE PER I PROCESSI SOFTWARE:

- Diversità
- Consegna
- Fiducia
  - creare documentazione, certificazioni, ecc. per far sì che l'utente (o un altro programmatore) possa fidarsi del software
  - ci sono enti standard che certificano la qualità del software
- Scala
  - avere un software che può essere scalato facilmente qualora il numero di utenti aumenti

PRINCIPI FONDAMENTALI:

- I metodi specifici, le tecniche e gli strumenti utilizzati dipendono dall'organizzazione che sviluppa il software, dal tipo di software e dalle persone coinvolte nel processo di sviluppo

- Non ci sono metodi di ingegneria del software universali che sono applicabili a tutti i sistemi e a tutte le aziende
- I concetti fondamentali dell'ingegneria del software sono **indipendenti dal linguaggio di programmazione** utilizzato per sviluppare il software

Il **processo** va sempre chiarito sin da subito, soprattutto quando si lavora in gruppo; altrimenti l'organizzazione e l'efficienza del gruppo collassa. Si devono definire sin da subito i ruoli, le responsabilità e le attività di ogni membro del gruppo, altrimenti il progetto fallisce

FIDATEZZA E PRESTAZIONI:
il software deve fare quello che deve fare, in maniera efficiente (e senza spreco di risorse), e in modo sicuro

REQUISITI:

- È importante capire e gestire la specifica e i requisiti del software
- Occorre sapere che cosa si aspettano clienti e utenti del sistema e occorre gestire le loro aspettative in modo che possa essere rilasciato un sistema utile entro i costi e i tempi previsti

RIUSO:
Riutilizzare vecchio codice è spesso più rapido ed economico di scrivere nuovo codice da zero, ma bisogna stare attenti che il codice riutilizzato non abbia bug o sia effettivamente adatto al nuovo contesto.

MODELLO DI PROCESSO:

- Ciascuna organizzazione utilizza il proprio processo
- Il modello di processo software è una rappresentazione semplificata e astratta che descrive l'intero ciclo di vita del software

"come Platone e il mondo delle idee"

Attività fondamentali dei processi software

- Ciascun processo è composto da attività fondamentali
- Tutti i processi software condividono queste attività fondamentali
- Tali attività possono essere organizzate e realizzate in modi diversi in processi diversi

4 ATTIVITÀ FONDAMENTALI DEI PROCESSI SOFTWARE:

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

STUDIO DI FATTIBILITÀ:

Stabilisce se lo sviluppo debba essere avviato, ossia:

- Se esiste un mercato per il software
- Se il software sia tecnicamente ed economicamente realistico

Definisce quali sono le alternative possibili e le scelte più ragionevoli, stimando le risorse (finanziarie e umane) necessarie per ciascuna alternativa

se in un progetto ci sono elementi già sviluppati da altri (se open source), ci permette di risparmiare tempo e denaro; se invece l'intero progetto è già stato sviluppato a un prezzo minore di quello che potremmo proporre, spesso è il caso di abbandonare il progetto.

Fornisce un report di fattibilità con:

- Definizione del problema
- Valutazione Costi/Benefici
- Risorse finanziarie e umane
- Soluzioni alternative
- Tempi di consegna e modalità di sviluppo

Tale studio dovrebbe essere **relativamente rapido** e **poco costoso** (se sto un mese a fare uno studio sui telefoni cellulari per capire quale acquistare e nel mentre esce una nuova generazione, ho buttato il mio tempo perché devo ricominciare da capo)

CODE AND FIX

Approccio alla produzione di software che consiste nello:

- Scrivere codice
- Aggiustare il codice per correggere errori, migliorare e/o aggiungere funzionalità

correggere errori (anche usando LLM per fare il debugging), uno ad uno, non è l'approccio ottimale per progetti di grande dimensioni, motivo per cui si necessita di approcci più organizzati allo sviluppo del software.

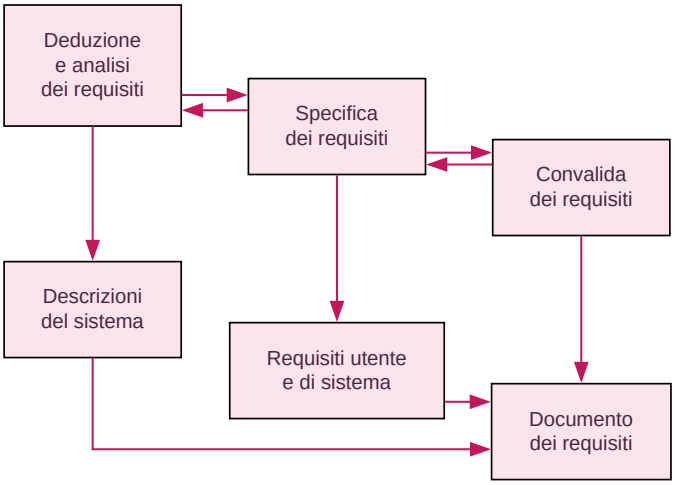
Acquisizione, analisi e specifica dei requisiti

- Attività per stabilire **cosa** il software dovrà fare
- Specificare funzionalità e qualità
- Definisce tramite l'interazione con il committente funzioni, vincoli, prestazioni, interfacce...
- Attività critica

bisogna dedurre e analizzare i requisiti del software, non solo da parte del cliente, ma anche da parte dei **portatori di interesse** (stakeholder):

- clienti
- committenti
- enti amministrativi esterni

bisogna avere un documento formale con cui si descrive in dettaglio quali sono le funzionalità e i vincoli nel progetto (che poi verranno convalidati da diversi enti)



La **deduzione** richiede spirito critico e può coinvolgere:

- osservazione di sistemi già esistenti
- discussione con i possibili utenti

La **specifica** traduce le informazioni dedotte in un insieme di requisiti

- Requisiti di **sistema**
- Requisiti **utente**

La **convalida** controlla che i requisiti siano realistici, coerenti e completi
Durante questo possono essere rilevati errori nel documento dei requisiti e in tal caso il documento dei requisiti deve essere modificato e corretto

Al termine di questo processo, se il progetto rispecchia gli standard, viene approvato.

Tutte queste attività sono interconnesse, perché in ogni fase si possono notare dei difetti o errori avvenuti in una fase precedente

Progettazione e Sviluppo

Divisa in due fasi:

- **progettazione**
- **sviluppo**

Le attività della fase di progettazione sono interconnesse e interdipendenti
I progettisti non ottengono immediatamente un risultato completo
Le nuove informazioni sul progetto influenzano le precedenti scelte progettuali

progettazione dell’architettura:

- definire classi, strutture, ecc.

progettare una interfaccia:

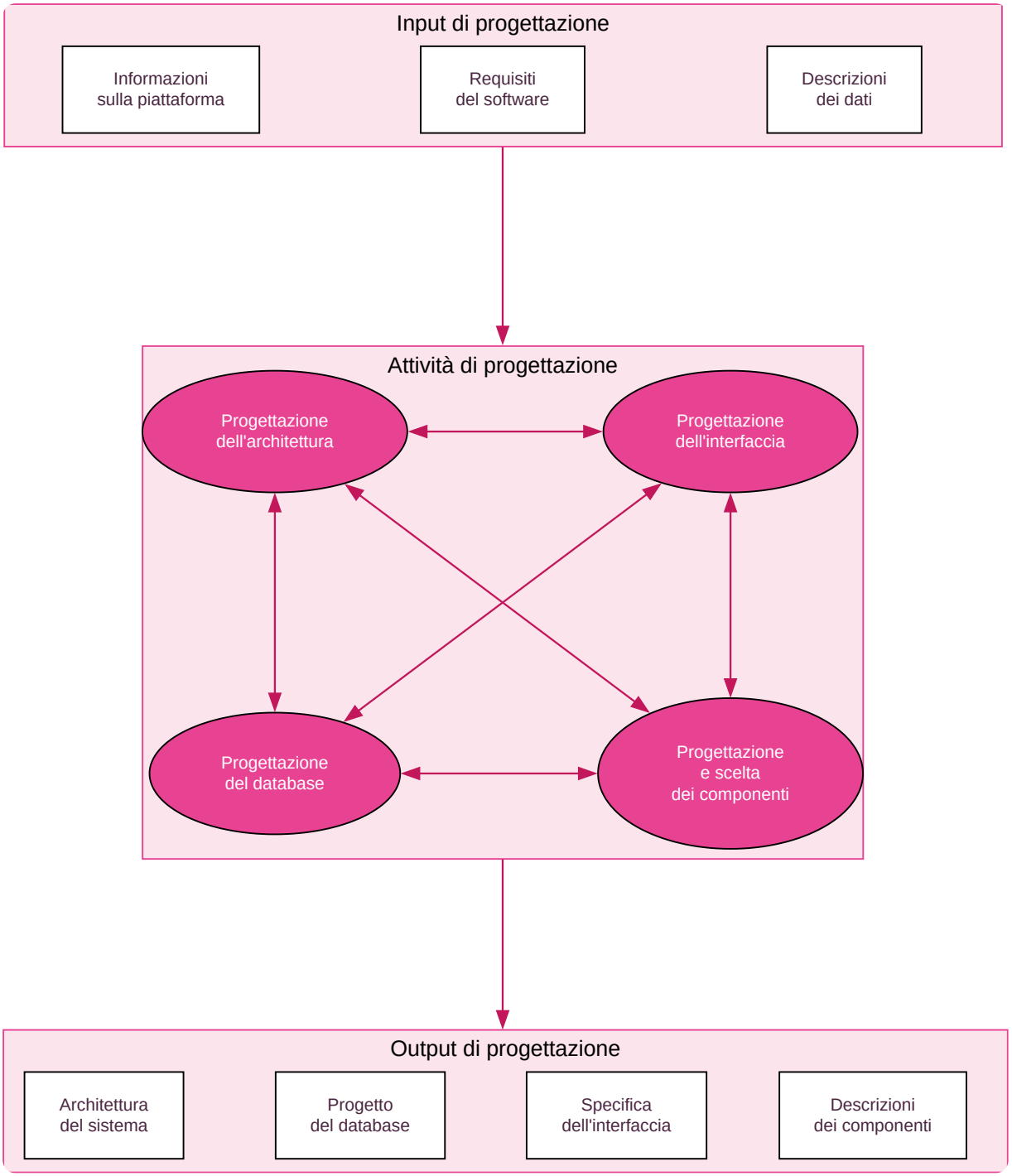
- non deve essere necessariamente una interfaccia grafica, ma deve essere in grado di soddisfare tutte le interazioni necessarie con l'esterno
- una volta definita una interfaccia, posso espandere il sw perchè è già suddiviso in moduli

Scelta dei componenti:

- avere componenti riutilizzabili e adattarli
- progettare componenti riutilizzabili
- bisogna mantenere i componenti flessibili, in modo che anche un altro sviluppatore possa riutilizzarli in un contesto diverso

l'output finale di queste fasi è un progetto software (formale o informale) che descrive:

- La struttura del software che si deve implementare
- I modelli e le strutture di dati usati dal sistema
- Le interfacce tra i componenti del sistema



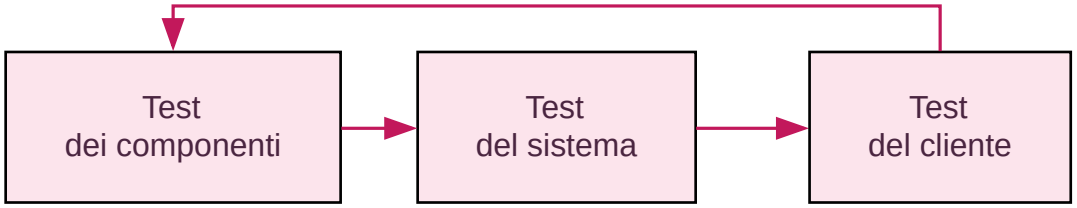
la parte di **sviluppo** è la meno importante in questo corso, perché è fortemente dipendente dalle restrizioni o indicazioni di una determinata azienda, cliente, ecc...

Verifica e Validazione

verifica: mostrare che un sistema è conforme alle sue specifiche
validazione: mostrare che un sistema soddisfa le esigenze del cliente

queste due fasi sono complementari tra loro, perché posso avere un ottimo software, ma se al cliente non piace, non è un buon software (o se non rispecchia determinate normative)
ogni volta che convoco clienti/stakeholder per la validazione, è un processo costoso, quindi è importante arrivare alla validazione con un software funzionante e che rispetti le aspettative.

per questo motivo abbiamo il **testing**:
a un software funzionante forniamo dei dati di prova (realistici) e si verifica che il software non fallisca e che i risultati siano conformi alle attese.



Test dei componenti

il componente può essere una funzione, una classe, un microservizio, ecc.
dobbiamo iniziare con il test del singolo componente (il fatto che tutti i componenti singolarmente funzionino non significa che il sistema complessivo funzioni)

un esempio di un sistema che funziona anche se un singolo componente non funziona:
una fotocamera che deve determinare la traiettoria di un oggetto in movimento, se un singolo componente è alterato (magari per una goccia di pioggia sul sensore), il modello del sensore fornisce un dato, ma il sistema, basandosi sul contesto e sugli altri sensori/dati, è in grado di correggere l'errore e fornire comunque un risultato corretto.

Test del sistema

una volta testati tutti i singoli componenti, si passa al test del sistema completo.
come accennato prima, se alcuni componenti funzionano bene, ma non sono stati scritti con interfacce compatibili (o altri problemi), il sistema nel complesso non funziona.
bisogna quindi fare del debugging e capire dove si trova il problema

Test del cliente

il sistema viene testato con i dati reali di un cliente in condizioni realistiche.
questo test può rilevare problemi con i requisiti
ci sono dei margini di tolleranza

Test iterativo

Alcuni errori nei componenti del programma possono apparire durante il test del sistema o il cliente può stimolare il sistema in modi non previsti dai dati simulati

Evoluzione

solitamente non bisognerebbe mai reinventare la ruota, ma è importante usare sistemi esistenti (e già testati esaustivamente) e modificarli per le proprie esigenze.
Con l'uso intensivo di LLM, si tende però a riscrivere librerie già esistenti da zero, perché la velocità con cui un LLM produce codice è paragonabile al cercare una libreria già esistente e adattarla alle proprie esigenze.

Nel corso della sua vita, il software viene modificato continuamente:

- manutenzione correttiva
- migliorie (efficienza o funzionalità)
- adattamento a nuovi mutamenti del contesto

Modelli di processi software

DEFINIZIONE modello del ciclo di vita del software (CVS):
Un modello del ciclo di vita del software è una caratterizzazione descrittiva o prescrittiva di come un sistema software viene o dovrebbe essere sviluppato

- (W. Scacchi in Encyclopedia of Software Engineering)

DEFINIZIONE STANDARD ISO/IEC/IEEE 12207:2026:
Software Life Cycle:
evolution of a system, product, service, project or other human-made entity from conception through retirement

Life Cycle Model:
framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding

nota: viene ancora definito **human-made entity** perchè anche se un LLM ha scritto il codice o ha contribuito nella fase di progettazione, è comunque un essere umano a dover validare il software e a doverlo rilasciare, quindi il software è sempre un prodotto umano.

Non esiste un processo universale.

Il modello mi astrae il processo di ciclo di vita reale di un software

nel '96 la definizione era molto più complessa:
non spiega neanche effettivamente a cosa servisse il modello ed aveva più righe di quella attuale

Ogni processo ha diverse possibili descrizioni:

- **Architetturale** – descrive la sequenza di attività senza fornire dettagli sulle specifiche attività (utilizzato in questo corso);
- **Data-flow** – evidenzia le trasformazioni dei dati operate dalle attività del processo;
- **Role/action** – I ruoli delle persone coinvolte nel processo e le relative responsabilità

perché si parla di ruoli e non di persone?
una persona può avere più ruoli, o un singolo ruolo può essere ricoperto da più persone.

la **descrizione** del processo sf include:

- Attività
- prodotti

- ruoli
- pre- e post-condizioni

Quale modello devo scegliere?

- **plan driven** (tutto deve essere pianificato in anticipo, e il processo deve seguire il piano)
- **agili** (non fortemente guidati da un piano, ma si adattano velocemente ai cambiamenti)

questi sono classi di modelli e bisogna sapere quali applicare in base al contesto, talvolta è anche solo preferenza personale.

nota: nel PW3 da pagina 13 ci sono un sacco di domande di teoria che spesso chiedono all'esame

modelli che vedremo in questo corso:

- modello a cascata
- modelli evolutivi
- modelli orientati al riuso
- modelli trasformazionali (cenni)

(sono tutti processi guidati da un piano, quindi plan driven)

Modello a cascata

mettiamo le varie attività in sequenza, la maggior parte delle attività degli artefatti prodotti in ogni fase e le **milestones** sono definite sin dall'inizio del progetto.
le fasi riflettono direttamente le attività fondamentali dei processi software:

- Definizione dei requisiti (produce lista dei requisiti)
- Progettazione del sistema e del software
- Implementazione e test delle unità (produce codice di test)
- Integrazione e test del sistema (produce test di sistema)
- operatività e manutenzione (sw finito integrato e testato)

la **documentazione** è fondamentale in questo modello, prodotto alla fine di ogni fase, ogni documento deve essere approvato prima di passare alla fase successiva.

se io ho fatto deigli errori nei requisiti se ne pagheranno le conseguenze molto tardi (i vari passaggi non possono essere modificati facilmente): non si può tornare indietro, perché il modello a cascata è **rigido**.
vantaggi:

- le fasi son ben definite (non ci sono soprese)
- gli output di ciascuna fase sono precisamente individuati

svantaggi (siamo nel mondo reale):

- essere così rigidi non è sempre fattibile (spesso il cliente stesso non sa cosa vuole)
- produce troppa documentazione (ora con gli LLM è più semplice, ma rimane comunque un processo lungo e costosos)
- poco flessibile: se succede un imprevisto bisogna tornare all'inizio e rifare tutto il processo

ma perchè si usa ancora il modello a cascata?
se ci sono degli enti che devono approvare il software, è necessario avere una documentazione completa e dettagliata (per esempio un sw che gestisce un aereo)

quando non usarlo?

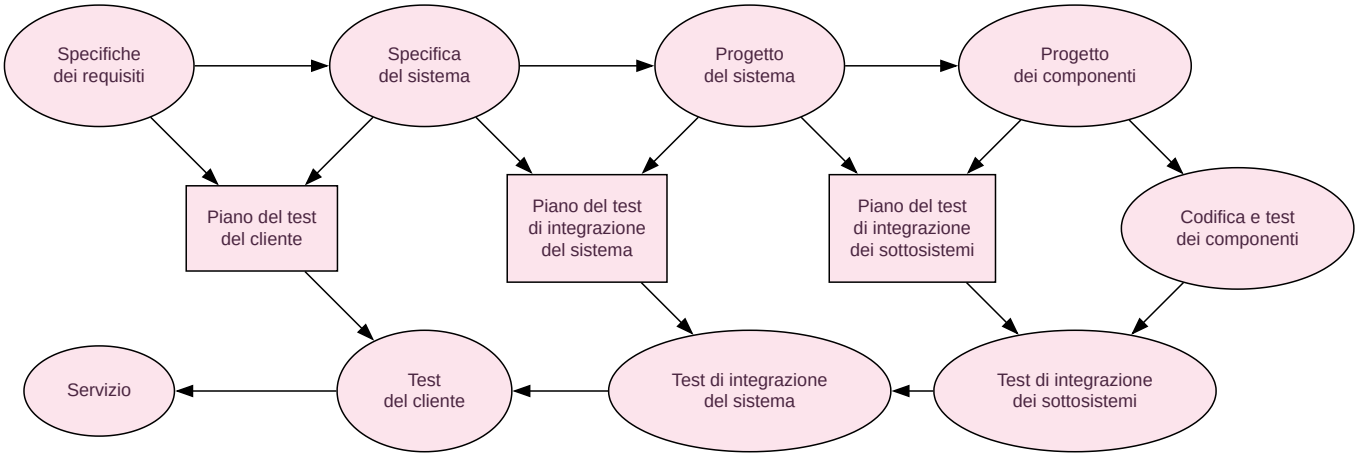
- con piccoli team di sviluppo
- con progetti non critici

Modello a cascata con retroazione

Il modello, ad ogni scambio di fase, si esegue anche uno scambio di opinioni e feedback, rendendo possibile tornare indietro e correggere eventuali errori nella fase precedente.
ogni volta che si torna indietro, la documentazione precedente deve essere aggiornata (il processo rimane estremamente formale)
il salto si può fare solo tra fasi adiacenti, ma si può tornare indietro più volte (Sempre aggiornando la documentazione)

Modello a cascata a V

è un modello molto più utilizzabile nel mondo reale:
per ogni fase di progettazione possiamo già andare a vedere con il cliente se i requisiti sono corretti o sono stati interpretati correttamente, così per ogni fase di sviluppo vengono realizzati dei test.



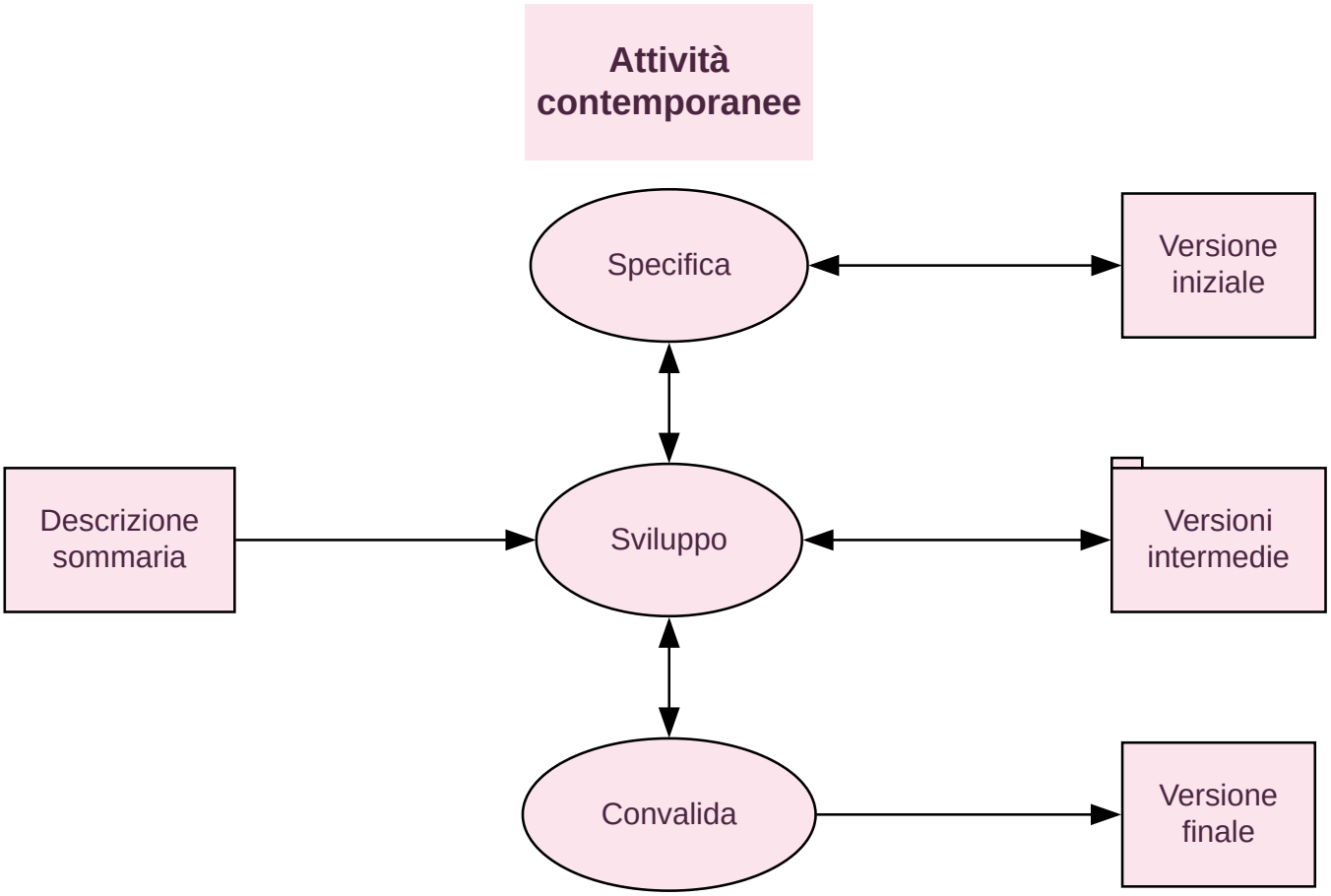
nei progetti grossi, il team di sviluppo è diverso dal team di testing, il team di testing deve cercare di fare "fallire" il sw o il piano in ogni modo possibile in ogni fase, così da poter correggere eventuali errori prima di passare alla fase successiva.

tutto deve essere sempre documentato ad ogni passaggio

Modelli evolutivi

- Modello a sviluppo/consegna crementale
  - lavorare con il cliente
- Modello prototipale
  - sviluppo di un prototipo per comprendere i requisiti del cliente

si inizia con una idea di base e si aggiungono elementi nel tempo, iniziando con dei requisiti iniziali relativamente vaghi, per poi arrivare a un prodotto finito e completo.



modello a sviluppo incrementale:
questo modello consente di avere feedback immediati e continui dal cliente:
al cliente non vengono forniti documenti (o vengon forniti pochi) ma solamente "esempi pratici" del sw, in questa maniera se al cliente non piace qualcosa, si può modificare il progetto senza dover rifare tutto il processo dall'inizio.

dopo diverse versioni intermedie, si arriva ad una versione finale, ed è quella che rilascerò al cliente

modello a consegna incrementale:
per essere sicuri che il sw funzioni esattamente come dovrebbe, vado ad installarlo nel

ambiente operativo del cliente man mano che lo sviluppo.

vantaggi:

- il cliente può usare il sw nel ambiente operativo reale
- feedback molto realistico e immediato

svantaggi:

- bisogna avere un cliente disponibile ad installare la demo sui sui sistemi e che abbia abbastanza tempo per testarla
- richiede parecchio tempo

ogni incremento rilascia delle priorità da implementare (definite con il cliente), una volta approvate vengono "congelate" e non si modificano più.
vado quindi a **mappare** i requisiti in base alla priorità:
copia grafico pw3 pag 36

sviluppo incrementale	consegna incrementale
la valutazione della prima versione è effettuata da un proxy degli utenti finali in un ambiente operativo diverso da quello target. Le versioni intermedie non sono solitamente rilasciate al cliente	permette una valutazione più realistica e rappresentativa dell'utilizzo reale del software perché ciascun incremento può essere rilasciato agli utenti finali nell'ambiente operativo del sistema

vantaggi:

- i problemi si trovano più rapidamente
- le funzionalità critiche, essendo state rilasciate subito, possono essere testate estensivamente (e hanno più feedback)

problemi:

- poca documentazione (manca visibilità del processo)
- difficile da mantenere per i continui cambiamenti
- codice spesso mal strutturato